

*Chaînes de caractères — Approfondissement***Exercice 1**

```
1 identite = "Martin Durand-Dupont"
2 init = identite[0]
3 for i in range(len(identite)):
4     if identite[i] == ' ' :
5         init = init + identite[i+1]
6 print(init.upper())
```

1. Décrire ce que fait le script ci-dessus à la ligne 2.
2. Que fait la boucle bornée associée à la condition de la ligne 4 ?
3. À quoi sert l'instruction de la ligne 6 du script ?
4. Qu'affiche le script lorsque l'utilisateur saisit :
 - a. martin durand ?
 - b. martin dupont durant ?
 - c. martin dupont-durant ?
5. Comment modifier le script pour que les réponses aux questions 4.b et 4.c soient identiques à celle de la question 4.b ?

Exercice 2

On propose le script ci-dessous.

```
1 phrase = "Donner une expression"
2 affiche = phrase[0]
3 print(affiche)
4 for i in range(1, len(phrase)):
5     affiche = affiche + phrase[i]
6     print(affiche)
```

1. Que fait le script à la ligne 2 ?
2. Expliquer ce que fait la boucle bornée.
3. Que renvoie le script lorsque l'utilisateur donne l'expression « Stylé » ?
4. On remplace `range(1, len(phrase))` par `range(len(phrase))` à la quatrième ligne du script. Que se passe-t-il ?

Exercice 3

On donne le script de la fonction *essai*.

```
1 def essai(n):
2     n = str(n)
3     return(len(n))
```

1. Quel est l'intérêt de l'instruction `str(n)` ?

2. Déterminer ce que renvoie la fonction pour les instructions suivantes.

- a. `essai(123)`
- b. `essai('Bonjour')`
- c. `essai(-596)`

Exercice 4 — Le code ASCII

L'*American Standard Code for Information Interchange* (Code américain normalisé pour l'échange d'information), plus connu sous l'acronyme *ASCII*, est une norme informatique de codage de caractères apparue dans les années 1960. C'est la norme de codage de caractères la plus influente à ce jour.

1. Saisir les instructions suivantes dans la console Python et compléter ce qui s'affiche pour chacune d'elles.

```
>>> ord('A')
.....
>>> ord('Z')
.....
>>> chr(65)
.....
```

2. Expliquer ce que font les fonctions `ord( )` et `chr( )`.

Exercice 5

Les commandes `upper( )`, `lower( )`, `capitalize( )`, `\t` et `\n`.

1. Saisir dans la console Python les instructions suivantes et compléter les affichages.

```
>>> phrase = 'La vie est belle'
>>> phrase.upper()
.....
>>> phrase.lower()
.....
>>> phrase.capitalize()
.....
```

2. En déduire le rôle des commandes `upper( )`, `lower( )` et `capitalize( )`.

3. Compléter le script ci-dessous qui demande à l'utilisateur son prénom, son nom et son âge, et affiche, en respectant les majuscules et minuscules, le résultat ci-dessous :

```
Salut Prénom NOM
tu as age ans
```

```
1 def identite(nom, prenom, age):
2     nom = .....
3     prenom = .....
4     age = str(age)
5     print("Salut", ....., ....., "\ntu as", ....., "
    ans")
```

4. Quelle est l'utilité des instructions `\t` et `\n` ?

Exercice 6 — Le chiffrement de César

Pour envoyer ses messages codés, Jules César utilisait une méthode de codage qui consiste à décaler chaque lettre de trois rangs dans l'alphabet. Ainsi, le A se transforme en D, le B en E, ..., et le Z en C.

1. a. Compléter le tableau en trouvant la lettre codée, puis en utilisant la fonction `ord()` pour déterminer le code ASCII.

Lettre en clair	C	E	S	A	R
Code ASCII					
Code ASCII + 3					
Lettre codée					

- b. Compléter la phrase :

Utiliser le code César revient à ajouter au code ASCII des lettres en clair du message.

2. À l'aide de la console Python, déterminer le code ASCII de l'espace.
3. On considère la fonction `code_cesar` qui reçoit un message et le renvoie codé.

```

1 def code_cesar(message) :
2     message = message.upper()
3     coder = ''
4     for caractere in message :
5         n = ord(caractere)
6         n_code = n + 3
7         lettre_c = chr(n_code)
8         coder = coder + lettre_c
9     return coder

```

- a. Expliquer ce que fait la boucle bornée.
 - b. Copier ou ouvrir le script dans l'éditeur Python et déterminer le codage des mots ci-dessous.
 - CESAR
 - ROYAUME
 - bizarre
 - c. Quel(s) problème(s) peut-on identifier ?
4. Compléter le script ci-dessous pour que le(s) problème(s) précédent(s) soi(en)t supprimé(s), puis écrire le codage du mot **ZYGOTE**.

```

1 def code_cesar(message) :
2     message = message.upper()
3     coder = ''
4     for caractere in message :
5         n = ord(caractere)
6         if .....:
7             .....
8         elif .....:
9             .....
10        else :
11            .....
12        lettre_c = chr(n_code)

```

```
13         coder = coder + lettre_c
14     return coder
```

5. **a.** Que faut-il faire pour décoder un message lorsque celui-ci est codé avec un décalage de 3 ?
- b.** Le message EUDYR YRXV HWHV IRUW a été codé avec un décalage de 3. Quel est le message en clair ?